

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ  
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ  
УНИВЕРСИТЕТ» (НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ, НГУ)

Факультет \_\_\_\_\_

Кафедра \_\_\_\_\_

Направление подготовки \_\_\_\_\_

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА**

\_\_\_\_\_  
(Фамилия, Имя, Отчество автора)

Тема работы \_\_\_\_\_

\_\_\_\_\_  
\_\_\_\_\_

**«К защите допущена»**

Заведующий кафедрой

ученая степень, звание

...../.....

(фамилия, И., О.) / (подпись, МП)

«.....».....20...г.

**Научный руководитель**

ученая степень, звание

должность, место работы

...../.....

(фамилия, И., О.) / (подпись, МП)

«.....».....20...г.

Дата защиты: «.....».....20...г.

Новосибирск, 20\_\_

# СОДЕРЖАНИЕ

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>ВВЕДЕНИЕ</b>                                           | <b>3</b>  |
| <b>1. Предметная область</b>                              | <b>6</b>  |
| 1.1. Функциональные промежуточные представления . . . . . | 6         |
| 1.1.1. Программирование в стиле передачи продолжений .    | 7         |
| 1.2. Императивные промежуточные представления . . . . .   | 7         |
| 1.2.1. CFG . . . . .                                      | 7         |
| 1.2.2. SSA . . . . .                                      | 7         |
| 1.2.3. Sea of Nodes . . . . .                             | 8         |
| 1.3. Соответствие между CPS и SSA . . . . .               | 8         |
| 1.4. Thoring IR . . . . .                                 | 8         |
| <b>2. РАЗРАБОТАННЫЙ ПОДХОД</b>                            | <b>9</b>  |
| <b>3. РЕЗУЛЬТАТЫ</b>                                      | <b>10</b> |
| <b>ЗАКЛЮЧЕНИЕ</b>                                         | <b>11</b> |
| <b>СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ</b>                   | <b>12</b> |
| <b>ПРИЛОЖЕНИЕ</b>                                         | <b>13</b> |
| П.1. Первая глава приложения . . . . .                    | 13        |

# ВВЕДЕНИЕ

Первые компиляторы появились во второй половине двадцатого века и прошли большой путь развития до наших дней. Сегодня компилятор - большая и сложная система, которой приходится мириться с новыми абстракциями языков программирования, появлением управляемых сред исполнения, с возросшей сложностью и разнообразием архитектур ЭВМ.

Чтобы такая большая программа как компилятор могла существовать, поддерживаться и развиваться, нужно упростить задачу, которую она решает, например, поделив её на подзадачи. Поэтому процесс компиляции принято делить на фазы, каждая из которых реализуется одним или несколькими программным модулями.

В настоящей работе нас будет интересовать модуль, осуществляющий *машинно-независимые* оптимизации, его мы будем называть *оптимизатором*. Машинно-независимые оптимизации - это трансформации программы, не требующие её отображение в машинный код целевой архитектуры.

Внутри оптимизатора программа находится в виде структуры данных, называемой *промежуточным представлением*. Такое название подчеркивает, что это лишь одно из состояний программы в её трансформации от исходника до эффективного машинного кода.

Как пользователи оптимизатора, мы хотим, чтобы он работал быстро и порождал корректные и эффективные программы. Как разработчики компилятора мы хотим чтобы с ним было удобно работать. Чтобы угодить всем нужно выбрать подходящее промежуточное представление. Как правило выбор промежуточного представления зависит от исходного языка,

именно он определяет набор применимых высокоуровневых оптимизаций, частоту их появления и необходимые анализы программы. А языки программирования бывают очень разными, что влечет за собой разнообразие промежуточных представлений.

Языки программирования принято делить по *парадигмам*, то есть по наборам абстракций, которые они предоставляют. Существуют *императивные*<sup>1</sup> и *функциональные* парадигмы программирования.

Основными абстракциями императивных языков программирования являются *последовательное исполнение выражений*, использование *управляющих конструкций*, наличие *переменных*, разбиение программ на *процедуры* или *функции*. Управляющие конструкции делают поток исполнения программы нетривиальным, чтобы представлять все возможные пути исполнения и анализировать поведения программы вдоль них используется классическое промежуточное представление программы в виде *графа потока управления*(англ. *Control Flow Graph, CFG*)[1]. Для упрощения анализа потока данных и некоторых оптимизаций вместе с CFG используется *SSA* форма программы[2]. Наследником идей этих представлений является *Sea of Nodes*, которое используется в современных промышленных статических и JIT-компиляторах для объектно-ориентированных языков программирования[3,4].

Основными абстракциями функциональных языков программирования являются *функциональные замыкания*, *функции высшего порядка*, *полиморфные функции*, *алгебраические типы данных*, *классы типов*. Языки этой парадигмы, такие как Haskell, SML, Scheme, выбирают в качестве про-

---

<sup>1</sup> К императивным языкам программирования мы будем относить и все *объектно-ориентированные*.

межуточного представления различные модификации лямбда-исчисления: Административную нормальную форму[5], Continuation Passing Style[6,7]. С помощью хорошо изученных трансформаций термов лямбда-исчисления можно эффективно описывать трансформации и оптимизации функциональных языков.

Современные языки программирования, однако, все более тяготеют к смешению парадигм, абстракции изначально появившиеся в функциональных языках перетекают и активно используются в современных объектно-ориентированных языках. Самым ярким примером такого заимствования являются функциональные замыкания, которые можно найти в любом современном императивном языке. Но промежуточные представления после подобных заимствований, часто остаются неизменными, и новые концепции выражаются в терминах исходного языка[8–10], что приводит к ухудшению результатов оптимизаций.

Отсюда возникает идея реализации *мультипарадигменного* промежуточного представления. В этой работе содержатся результаты разработки функционального расширения императивного промежуточного представления и анализ его эффективности в рамках экспериментального компилятора Huawei.

# 1. Предметная область

## 1.1. Функциональные промежуточные представления

TODO

В качестве *промежуточного представлений* оптимизаторов функциональных языков программирования используются различные модификации *лямбда-исчисления*(1.1).

$$term ::= var \mid (term \ term) \mid (\lambda var. term) \quad (1.1)$$

Классические преобразования термов лямбда исчисления, такие как  *$\beta$ -редукция*,  *$\eta$ -конверсия* подходят для описания большинства оптимизаций функциональных языков. Посмотрим на терм  $(\lambda x. 0) (f y)$ , применение к нему шага  $\beta$ -редукции соответствует удалению избыточных вычислений<sup>2</sup>.

Набор подобных преобразований термов в оптимизаторе называется *правилами переписывания* термов. Например правило для удаления лишних вычислений можно записать так

$$(\lambda x. M)N \rightarrow M \text{ } x \notin FV(M)$$

На практике в оптимизаторах часто используется *типизированное лямбда исчисление*, с добавленным связыванием переменных (англ. let-bindings) и сопоставлением с образцом (англ. pattern-matching)[5].

Для знакомства с функциональными *промежуточными представления-*

---

<sup>2</sup>Конечно, такая бета-редукция применима только к ленивым языкам, т.к. вычисление терма  $(f y)$  может привести к исключению или к незавершающемуся исполнению.

ми нам будет достаточно классического *лямбда исчисления* со связыванием переменных.

$$(\lambda x. M)N \equiv \text{let } x = N \text{ in } M$$

### 1.1.1. Программирование в стиле передачи продолжений

TODO

Программирование в стиле передачи продолжений (англ. continuation passing style, CPS), это шаблон программирования, позволяющий сделать поток управления программы явным, используя для этого функции высшего порядка, которые мы будем называть *продолжением* (англ. *continuation*). Будем говорить, что функция находится в *cps*, если:

1. Ну принимает продолжение тип, ещё там тип есть  $(t \rightarrow' a)$ , где  $t$  - это известный тип, а  $'a$  - некоторая типовая переменная.
2. Функция является хвостовой рекурсией.

## 1.2. Императивные промежуточные представления

### 1.2.1. CFG

TODO

### 1.2.2. SSA

TODO

### **1.2.3. Sea of Nodes**

TODO

## **1.3. Соответствие между CPS и SSA**

TODO Поможет при описании преобразования Thorin'a обратно в SSA

## **1.4. Thoring IR**

TODO



## **2. РАЗРАБОТАННЫЙ ПОДХОД**

TODO

### **3. РЕЗУЛЬТАТЫ**

TODO

## **ЗАКЛЮЧЕНИЕ**

TODO

# СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Владимиров К. Оптимизирующие компиляторы. Структура и алгоритмы. Litres, 2024.
2. Rastello F., Tichadou F.B. SSA-based compiler design. Springer, 2022.
3. Click C., Paleczny M. A simple graph-based intermediate representation // ACM Sigplan Notices. ACM New York, NY, USA, 1995. Т. 30, № 3. С. 35–49.
4. Duboscq G. и др. Graal IR: An extensible declarative intermediate representation // Proceedings of the Asia-Pacific Programming Languages and Compilers Workshop. 2013. С. 1–9.
5. Jones S.L.P., Santos A.M. A transformation-based optimiser for Haskell // Science of computer programming. Elsevier, 1998. Т. 32, № 1-3. С. 3–47.
6. Appel A., Jim T. Continuation-passing style, closure-passing style // 16th Ann. ACM Symp. on Principles of Programming Languages. 1989.
7. Adams N. и др. Orbit: An optimizing compiler for Scheme // ACM SIGPLAN Notices. ACM New York, NY, USA, 1986. Т. 21, № 7. С. 219–233.
8. LLVM Project. LLVM Language Reference Manual. 2025.
9. Gosling J. и др. The Java Language Specification, Java SE 8 Edition. Addison-Wesley, 2015.
10. Evans B. Understanding Java method invocation with invokedynamic // Java Magazine. 2021.

# **ПРИЛОЖЕНИЕ**

## **П.1. Первая глава приложения**

TODO